The Hospital Management System is designed to simplify and streamline hospital operations while ensuring effective healthcare delivery. It focuses on managing patient records, handling emergency cases, tracking treatment history, and assigning doctors efficiently. The system allows easy addition, viewing, updating, and deletion of patient data. In emergency situations, patients are prioritized based on the severity of their condition to ensure quick medical response. Each patient's treatment history is recorded using a stack structure, and doctor assignments are handled quickly through a hash table for fast lookup. Behind the scenes, the system utilizes data structures such as linked lists, priority queues, stacks, and hash tables to maintain organized operations and support scalability as the hospital grows. This leads to better patient care and more efficient use of resources.

-Description of Data Structures Used:

Testing Process Explanation:

The Hospital Management System was tested in different ways to ensure everything works correctly and efficiently:

1.Unit Testing – Each part of the system, including patient records, emergency queue, treatment history, and doctor assignments, was tested separately to make sure it functions properly.

2. Integration Testing – All parts of the system were brought together and tested through the Main class to make sure they work well together and the whole system runs smoothly.

-Description of Data Structures Used:

1. Patient Records – Singly Linked List:

To store and manage patient information, a singly linked list is used. Each node in the list represents one patient and contains details like Patient ID, Name, Age, Contact Info, and Medical History.

This structure is helpful because it allows us to add or remove patients easily without needing the memory to be next to each other. In Java, we can create a Patient class to store the data, and another class like PatientRecords to handle list operations such as insert, delete, and search.

2. Emergency Queue – Priority Queue:

For emergency cases, patients need to be treated based on how serious their condition is and when they arrived. That's why a **priority queue** is the best fit.

In Java, we can use the PriorityQueue class with a **custom comparator** to make sure that patients with higher severity come first. If two patients have the same severity, the one who arrived earlier will be treated first.

3. Treatment History – Stack:

To keep track of each patient's treatment history, we use a **stack**. This way, new treatments are added on top, and we can easily check the latest one. In Java, we can use the built-in Stack class for this purpose.

4. Doctor Assignments – Hash Table:

To manage doctor information quickly and efficiently, we use a **hash table**. In Java, the HashMap class is perfect for this. Each doctor's ID can be linked to their details (like name, specialty, or assigned patients), making it easy to access or update info when needed.

Test Patient Records:

- Add two patients with the following details:
  - Patient 1: ID=101, Name=Ghadi, Age=21, Contact=[demo]
  - Patient 2: ID=102, Name=Joud, Age=20, Contact=[demo]
- Display all patients.

Test Emergency Queue:

- Add patients Ghadi and Joud to the queue.
  - Ghadi (Less Critical - Priority 3)
  - Joud (More Critical - Priority 1)
- Retrieve and treat the highest-priority patient.

Test Treatment History:

- Add three treatment records:
  - " Amoxicillin 500mg"
  - " Panadol Extra"
  - " Vitamin D3"
- Display the full treatment history.
- Retrieve the last treatment performed.

Test Doctor Assignments:

- Add two doctors:
  - Doctor 1: ID=201, Name=Dr. Rana, Department= Psychiatry, Schedule=8AM-6PM
  - Doctor 2: ID=202, Name=Dr. Sahar, Department=Neurology, Schedule=11AM-8PM

- Display all doctors.

Integration Testing (Main class):

1. Created and managed patient records.
2. Simulated emergency queue operations.
3. Managed treatment history for patients.
4. Assigned and retrieved doctor information.

Source Code:

```
import java.util.*;

public class Patient implements Comparable<Patient> {

    int id;

    String name;

    int age;

    String contact;

    LinkedList<String> medicalHistory;

    int severity;

    Patient next;


    public Patient(int id, String name, int age, String contact) {

        this.id = id;

        this.name = name;

        this.age = age;

        this.contact = contact;

        next = null;

        medicalHistory = new LinkedList<>();
```

```
    }
    public void addMedicalHistory(String str){
        medicalHistory.add(str); //We add the string to the linked list
    }
    @Override
    public int compareTo(Patient p) {
        return this.severity-p.severity;
    }
    @Override
    public String toString(){
        return "ID: " + id +", Name: " + name+", Age: " + age +", Contact: " +
contact
                +"\nMedical History: "+medicalHistory;
    }
}
```

```
public class PatientRecords {
    private Patient first;
    public PatientRecords() {
      first = null;
    }
    public void addPatient(Patient p) {
     Patient newPatient = p;
      if (first == null)
      {
        first =  p;
```

```
        }
        else
        {
            Patient current = first;
            while (current.next != null)
            {
                current = current.next;
            }
            current.next = p;
        }
    }
    public void updatePatient(int id, String name, int age, String contact) {
        Patient current = first;
        while (current != null) {
            if (current.id == id) {
                current.name = name;
                current.age = age;
                current.contact = contact;
                System.out.println("Patient has been modified.");
                return;
            }
            current = current.next;
        }
        System.out.println("Patient with ID " + id + " is unavailable.");
    }
```

```
public void removePatient(Patient p) {


 Patient current = first;
    Patient previous = null;


    if (current != null && current.id == p.id) {
        first = current.next; // Changed head
        System.out.println("Patient with ID " + p.id + " has been deleted.");
        return;
    }
    while (current != null && current.id != p.id) {
        previous = current;
        current = current.next;
    }


    if (current == null)
    {
        System.out.println("Patient with ID " + p.id + " is unavailable.");
        return;
    }
    previous.next = current.next;
    System.out.println("Patient with ID " + p.id + " has been deleted.");
}
public void displayPatients() {
```

```
        Patient current = first;
        while (current != null) {
            System.out.println(current.toString());
            current = current.next;
          }
      }
  }
```

```
import java.util.*;
public class EmergencyQueue {
   private PriorityQueue<Patient> queue;
   public EmergencyQueue() {
      // Initialize the priority queue
      queue = new PriorityQueue<>();
   }
   public void addPatient(Patient patient, int severity) {
      // Set severity and add patient to the queue
      patient.severity = severity;
      queue.add(patient);

   public Patient treatPatient() {
      if (queue.isEmpty()) {
         System.out.println("Emergency queue is empty.");
         return null;
      }
      //Treat and remove the highest priority patient
```

```
        return queue.poll();
    }
     public void displayQueue() {
        for (Patient p : queue) {
            System.out.println("ID: " + p.id +", Name: " + p.name);
        }
    }
}
```

```
import java.util.Stack;
public class TreatmentHistory {
    private Stack<String> treatments;

    public TreatmentHistory() {
        // Create the stack for treatments
        treatments = new Stack<>();
    }

    public void addTreatment(String treatment) {
        // Add a new treatment to the history
        treatments.push(treatment);
    }
    public String getLastTreatment() {
        if (treatments.isEmpty()) {
            return "No treatments recorded.";
        }
```

```
        return treatments.peek();
    }

    public void displayHistory() {
        // Show the treatment history
        System.out.print("Treatment History: "+treatments);
    }
}
```

```
public class Doctor {
    //Variables to store doctor info
    int id;
    String name;
    String department;
    String schedule;
    // Constructor to set doctor details
    public Doctor(int did, String dname, String department, String schedule) {
        this.id = did;
        this.name = dname;
        this.department = department;
        this.schedule = schedule;
    }

    // Format doctor info as a string
    @Override
    public String toString() {
```

```
        return "Doctor ID: " + id + ", Name: " + name +
                ", Department: " + department + ", Schedule: " + schedule;
    }
}
```

```
import java.util.*;
public class DoctorAssignments {
    // Create a table to store doctors
    private HashMap<Integer, Doctor> doctors;

    public DoctorAssignments() {
        doctors = new HashMap<>();
    }
    // Add a new doctor to the table
    public void addDoctor(Doctor doctor) {
        doctors.put(doctor.id, doctor);
    }
    public Doctor getDoctor(int did) {
        return doctors.get(did);
    }
    // Print all assigned doctors
    public void displayDoctors() {
        if (doctors.isEmpty()) {
            System.out.println("No doctors in the HashMap.");
            return;
        }
```

```
        for (Doctor doctor : doctors.values()) {
            System.out.println(doctor.toString());
        }
    }
}
```

```
public class Main {
    public static void main(String[] gargs) {
        PatientRecords patientRecords = new PatientRecords();
        System.out.println("Testing Patient Records...");
        Patient p1 = new Patient(101, "Ghadi", 21, "[demo]              ");
        Patient p2 = new Patient(102, "Joud", 20, "[demo]              ");
        patientRecords.addPatient(p1);
        patientRecords.addPatient(p2);
        patientRecords.displayPatients();

        System.out.println("\nTesting Emergency Queue...");
        EmergencyQueue emergencyQueue = new EmergencyQueue();
        emergencyQueue.addPatient(p1, 3);
        emergencyQueue.addPatient(p2, 1);
        System.out.println("Treating Patient: " + emergencyQueue.treatPatient().name);

        System.out.println("\nTesting Treatment History...");
        TreatmentHistory history = new TreatmentHistory();
        history.addTreatment("Amoxicillin 500mg");
        history.addTreatment("Panadol Extra");
        history.addTreatment("Vitamin D3");
        history.displayHistory();
        System.out.println("\nLast Treatment: " + history.getLastTreatment());

        System.out.println("\nTesting Doctor Assignments...");
        DoctorAssignments doctorAssignments = new DoctorAssignments();
        doctorAssignments.addDoctor(new Doctor(201, "Dr. Rana", "Psychiatry", "8AM-6PM"));
        doctorAssignments.addDoctor(new Doctor(202, "Dr. Sahar", "Neurology", "11AM-8PM"));
```

```
        doctorAssignments.displayDoctors();
    }
}
```

Screenshots of Output:

```
Output from recovered Java sources

Testing Patient Records...
ID: 101, Name: Ghadi, Age: 21, Contact: 00000000001
Medical History: []
ID: 102, Name: Joud, Age: 20, Contact: 00000000002
Medical History: []

Testing Emergency Queue...
Treating Patient: Joud

Testing Treatment History...
Treatment History: [Amoxicillin 500mg, Panadol Extra, Vitamin D3]
Last Treatment: Vitamin D3

Testing Doctor Assignments...
Doctor ID: 201, Name: Dr. Rana, Department: Psychiatry, Schedule: 8AM-6PM
Doctor ID: 202, Name: Dr. Sahar, Department: Neurology, Schedule: 11AM-8PM
```